

权值并查集解决鱼类合并问题

<https://gemini.google.com/app/c4dddb086f74906a>

User prompt: 这题怎么用权值并查集做

Response: 这道题的核心突破口在于题目中的一个关键条件：“保证 v 不小于之前放入过的所有鱼的大小”。

这意味着鱼是按照**大小非降**的顺序被加入网格的。因此，每当加入一条新鱼时，它不仅是当前连通块中最新、最大的鱼，还能直接吃掉四周所有已存在的邻居鱼。这就带来了一个非常漂亮的树状拓扑结构，可以通过带路径压缩的权值并查集（Weighted DSU）来完美解决。

核心物理量与转化

我们可以把每次放入新鱼的过程看作是在构建一棵 **Kruskal 重构树**：

- 每条新鱼放入时，如果它挨着某些已有的连通块，它就会成为这些连通块根节点的**双亲（Parent）**。
- 由于新鱼的大小 $\geq$ 之前的任何鱼，它一出生就能直接把这些子树里的鱼全吃掉。

对于一个已经在树中的老节点 u ，如果它想借着后续不断加入的新鱼“游向更远的世界”（即扩大连通块），它必须能够一路向上走，跨越每一层父节点。假设 u 所在的某个历史连通块根节点为 x ，现在它被合并到了一个大小为 v 的新父节点 p 上：

1. u 想要跨过 x 进入 p ，它必须先吃完整个 x 子树内的所有鱼来壮大自己。
2. 吃完 x 子树后， u 的体型会增加 $sz[x] - 1$ （其中 $sz[x]$ 为 x 子树内的总鱼数）。
3. 此时 u 的体型为 $V_{init} + sz[x] - 1$ 。为了能吃掉父节点 p ，必须满足：

$$V_{init} + sz[x] - 1 \geq v \implies V_{init} \geq v - sz[x] + 1$$

因此，从子树根节点 x 跨越到父节点 p 的体型门槛（边权）为：

$$W(x) = v - sz[x] + 1$$

由于每条鱼只会被合并一次，一旦它从根节点变成了某个新节点的儿子，**这条边权 $W(x)$ 就永远固定了！**

权值并查集设计

我们用一个权值并查集来维护每个节点到它当前所属的大连通块根节点的路径信息。

- `parent[i]`：节点 `i` 的父亲节点。
- `max_w[i]`：从节点 `i` 到 `parent[i]` 的路径上，最大的“门槛权值” W 。
- `sz[i]`：以 `i` 为根的子树大小（仅对当前是根节点的 `i` 有效）。

路径压缩（Find 操作）

在路径压缩时，我们不仅更新父亲指向，同时利用邻项求最大值的方式更新 `max_w`：

```
int find(int i) {
    if (parent[i] == i) return i;
    int root = find(parent[i]);
    max_w[i] = max(max_w[i], max_w[parent[i]]);
    return parent[i] = root;
}
```

具体操作实现

操作 1：在 (x, y) 处放入大小为 v 的新鱼

1. 解密坐标，为当前位置新建一个节点 `id`。初始化 `parent[id] = id, sz[id] = 1, max_w[id] = -INF`。

2. 检查 (x, y) 四周的邻居。如果邻居有鱼，通过 `find` 找到邻居所在连通块的当前根节点 `r`。
3. 过滤出所有去重后的独特邻居根节点（避免重复合并同一个块）。
4. 对于每个邻居根节点 `r`：
 - 记录边权： $\text{max_w}[r] = v - \text{sz}[r] + 1$
 - 改变父亲： $\text{parent}[r] = \text{id}$
 - 累加子树大小： $\text{sz}[\text{id}] += \text{sz}[r]$
5. 答案：当前新鱼能吃掉的最大鱼数就是新连通块的总鱼数减去它自己，即 $\text{sz}[\text{id}] - 1$ 。

操作 2：查询 (x, y) 处的鱼吃饱所需的最小增量

1. 解密坐标，找到该位置对应的节点 `id`。
2. 调用 `find(id)`。此时，`id` 的父亲已经被压缩指向了当前整张网格的绝对根节点 `R`，而 $\text{max_w}[\text{id}]$ 存储的就是从 `id` 到 `R` 路径上的最高阈值。
3. 要想吃掉最多数量的鱼（即达到整个连通块的上限 $\text{sz}[R] - 1$ ），这条鱼的初始体型最低必须达到 $\max(\text{size}[\text{id}], \text{max_w}[\text{id}])$ 。
4. 答案：所需的最小增加量即为 $\max(0, \text{max_w}[\text{id}] - \text{size}[\text{id}])$ 。

关键代码框架

```
#include <iostream>
#include <vector>
#include <algorithm>

using namespace std;
```

```

const int INF = 2e9;

struct DSU {
    vector<int> parent;
    vector<int> max_w;
    vector<int> sz;
    vector<int> val; // 记录每条鱼的初始大小 v

    DSU(int n) {
        parent.resize(n + 1);
        max_w.assign(n + 1, -INF);
        sz.assign(n + 1, 1);
        val.resize(n + 1);
        for (int i = 0; i <= n; i++) parent[i] = i;
    }

    int find(int i) {
        if (parent[i] == i) return i;
        int root = find(parent[i]);
        max_w[i] = max(max_w[i], max_w[parent[i]]);
        return parent[i] = root;
    }

    void merge(int u, int v_parent, int val_p) {
        int root_u = find(u);
        if (root_u != v_parent) {
            max_w[root_u] = val_p - sz[root_u] + 1;
            parent[root_u] = v_parent;
            sz[v_parent] += sz[root_u];
        }
    }
};

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

```

```

int n, m, q;
if (!(cin >> n >> m >> q)) return 0;

// 用一个二维数组记录网格中每个位置对应的鱼的节点编号, 0表示障碍
vector<vector<int>> grid(n + 1, vector<int>(m + 1, 0));
DSU dsu(q + 1); // 最多会有 q 条鱼被创建

int fish_cnt = 0;
int last_ans = 0;

int dx[] = {-1, 1, 0, 0};
int dy[] = {0, 0, -1, 1};

for (int i = 0; i < q; i++) {
    int op;
    cin >> op;
    if (op == 1) {
        int x_prime, y_prime, v;
        cin >> x_prime >> y_prime >> v;
        int x = x_prime ^ last_ans;
        int y = y_prime ^ last_ans;

        fish_cnt++;
        grid[x][y] = fish_cnt;
        dsu.val[fish_cnt] = v;

        vector<int> neighbors;
        for (int d = 0; d < 4; d++) {
            int nx = x + dx[d], ny = y + dy[d];
            if (nx >= 1 && nx <= n && ny >= 1 && ny <= m && grid[nx][ny] > 0) {
                neighbors.push_back(dsu.find(grid[nx][ny]));
            }
        }
        // 去重, 防止同一个连通块被多次合并导致 sz 重复计算
        sort(neighbors.begin(), neighbors.end());
        neighbors.erase(unique(neighbors.begin(), neighbors.end()), neighbors.end());
    }
}

```

```

        for (int r : neighbors) {
            dsu.merge(r, fish_cnt, v);
        }

        last_ans = dsu.sz[fish_cnt] - 1;
        cout << last_ans << "\n";

    } else if (op == 2) {
        int x_prime, y_prime;
        cin >> x_prime >> y_prime;
        int x = x_prime ^ last_ans;
        int y = y_prime ^ last_ans;

        int id = grid[x][y];
        dsu.find(id); // 触发路径压缩更新 max_w

        int req_v = max(dsu.val[id], dsu.max_w[id]);
        last_ans = max(0, req_v - dsu.val[id]);
        cout << last_ans << "\n";
    }
}
return 0;
}

```

通过这种设计，单次操作的时间复杂度被完美控制在了并查集的几乎常数级别 $O(\alpha(q))$ ，总时间复杂度为 $O(q \cdot \alpha(q))$ ，面对 5×10^5 的数据量可以在 0.1 秒内轻松跑完。
